

Usability Report

Purpose

This tool provides a conversational interface for inspecting QoR metrics from LibreLane/OpenROAD runs, diagnosing timing, area, power, and violation issues, and recommending tuning actions. Everything is based on evidence from parsed reports.

Intended User

EDA researchers, hardware design students, or architects running LibreLane on benchmarks who want quick, evidence-based guidance about QoR without manually reading multi-corner timing reports.

How to Run

Prerequisites: Python 3.8 or later. No third-party packages required.

Step 1 - Parse reports:

```
python scripts/parse_existing_runs.py
```

This produces results/qor_metrics.csv and results/parsed_summary.json.

Step 2 - Launch assistant:

```
python scripts/assistant.py
```

Step 3 (optional) - Run evaluation:

```
python evaluation/evaluate.py
```

Step 4 - Full validation:

```
python scripts/validate_project.py
```

Supported Interactions

Command	What it does
summary	Shows run count, clock periods, timing status, fastest setup-clean run
best run	Selects smallest CLOCK_PERIOD among timing-clean runs; notes
blockers	
why timing bad	Diagnoses setup/hold failures across all runs
explain run clock_25	Detailed per-run analysis with all metrics
compare runs	Side-by-side table of all runs (or specific requested runs)
what should I tune	Blocker-priority recommendation using 13-level hierarchy
qor status	QoR classification with Pareto analysis
what to fix first	Priority-ordered blocking issues and secondary opportunities

show metrics	Full metrics display per run
violations	Slew, cap, fanout violation details
power	Power breakdown (total, internal, switching, leakage)
area	Area and utilization with density analysis
congestion	Routing quality (DRC vs overflow distinction)
explain all	Natural-language explanation of every run
help	Lists all available commands

Strengths

1. Zero dependencies - runs with Python standard library only.
2. Offline operation - no internet or API keys required for core functionality.
3. Evidence-based - every answer cites metric values and source files.
4. No fabrication - missing data is explicitly reported as missing.
5. Correct WNS/WS distinction - never labels positive WS margin as positive WNS.
6. Structured output - diagnosis objects have severity, category, finding, evidence, and recommendation.
7. Dual mode - interactive REPL and one-shot command-line.
8. Multi-corner awareness - parser extracts per-corner timing and identifies worst corners.
9. Configurable thresholds - timing margin categories and utilization thresholds are adjustable.
10. Optional LLM - enhanced explanations available without changing core reliability.

Usability Evaluation

Task completion:

Task	Achievable?	Notes
Determine if timing is met	Yes	Uses normalized setup_slack (WS when WNS=0, WNS when negative)
Find fastest setup-clean run	Yes	Selects smallest clock among timing-clean runs; notes blockers
Understand why timing fails	Yes	Explains negative WNS with evidence
Get tuning recommendation	Yes	Rules-based on setup_slack with conditional language
Check violations	Yes	Reports slew/cap/fanout counts with worst corner info
Analyze power	Yes	Full breakdown if data available
Check congestion	Partially	Reports DRC, wirelength, vias; does not infer congestion from zero DRC
Compare two runs	Yes	Side-by-side table output
Understand WNS vs WS	Yes	Shows both separately with correct labels

Timing logic detail:

Timing pass/fail and tuning decisions use normalized setup_slack. If setup WNS is negative, setup_slack uses WNS (timing failure). If setup WNS is zero and setup WS is available,

setup_slack uses the positive WS margin. This normalized slack drives all threshold-based diagnosis.

Multi-corner support:

The parser extracts corner-specific timing values from OpenROAD JSON, computes normalized per-corner slack, identifies the worst setup and hold corners, and reports global violation counts alongside worst-corner identification. It does not perform detailed path-group analysis.

Response quality:

All responses are derived from parsed data. 24 out of 24 evaluation scenarios pass. Repository-relative source file paths are included in answers. Missing data is explicitly flagged and no values are hallucinated. Run `python scripts/validate_project.py` for the current test count and validation status.

Limitations

1. CLI only - no web interface or graphical visualization.
2. Partial dataset - 4 out of 10 configurations have complete report artifacts; 2 failed execution; 4 are config-only.
3. Single benchmark - PM32 only.
4. Stateless intent matching - question routing uses keyword/pattern matching, not NLU. Multi-turn context is minimal (tracks selected run and last comparison for pronoun resolution).
5. No congestion quantification - reports routing quality data but does not infer a congestion percentage when overflow metrics are unavailable.
6. No path-group analysis - timing diagnosis uses overall and per-corner slack, not path-group detail.
7. Static data - assistant loads metrics at startup with no live reloading.
8. Auto-tuning is dry-run only - without LibreLane binary, tuning demonstrates the workflow but cannot execute actual runs.

Comparison: Rule-Based vs LLM Mode

Aspect	Rule-Based	LLM Mode
Speed	Instant	2-5 seconds per query
Reliability	Deterministic	May vary
Fabrication risk	Zero	Constrained by validation logic
Internet required	No	Yes (API call)
Cost	Free	API token cost
Explanation depth	Template-based	Contextual natural language
Recommended for	Evaluation, automation	Human exploration